

Kiến trúc hiện đại

1. Microservices Architecture

- **Khái niệm:** Hệ thống được chia thành nhiều dịch vụ nhỏ (microservices), mỗi dịch vụ đảm nhiệm một *bounded context* riêng, triển khai độc lập, giao tiếp thường qua API REST, gRPC hoặc messaging.
- **Ưu điểm:**
 - ✓ Triển khai và mở rộng độc lập từng dịch vụ.
 - ✓ Dễ dàng áp dụng công nghệ khác nhau cho từng microservice.
 - ✓ Đội phát triển có thể làm việc song song, tăng năng suất.
- **Thách thức:**
 - ✓ Quản lý phức tạp (CI/CD, logging, monitoring, service discovery).
 - ✓ Giao tiếp phân tán có độ trễ, dễ phát sinh lỗi (network overhead).
 - ✓ Quản lý dữ liệu phân tán (eventual consistency) khó khăn.
- **Ví dụ:** E-commerce (Amazon, Shopee), ứng dụng gọi xe (Grab, Uber).

2. Serverless Architecture (FaaS)

- **Khái niệm:** Không quản lý server trực tiếp. Chỉ viết các *functions* chạy trong môi trường cloud (AWS Lambda, Azure Functions, Google Cloud Functions).
- **Ưu điểm:**
 - ✓ Tiết kiệm chi phí (pay-per-use).
 - ✓ Tự động scale theo nhu cầu.
 - ✓ Tập trung vào logic nghiệp vụ, không lo hạ tầng.
- **Thách thức:**
 - ✓ Cold start làm tăng độ trễ khi khởi chạy lần đầu.
 - ✓ Giới hạn thời gian chạy, bộ nhớ.

- ✓ Vendor lock-in (phụ thuộc nhà cung cấp cloud).
 - **Ví dụ:** Xử lý ảnh upload, chatbot, IoT event handling.
-

3. Event-Driven Architecture (EDA)

- **Khái niệm:** Thành phần trong hệ thống giao tiếp qua *sự kiện* (event). Producer phát sự kiện, consumer lắng nghe và phản ứng.
 - **Ưu điểm:**
 - ✓ Loose coupling → thành phần ít phụ thuộc lẫn nhau.
 - ✓ Hệ thống linh hoạt, dễ mở rộng.
 - ✓ Phù hợp xử lý bất đồng bộ, real-time.
 - **Thách thức:**
 - ✓ Khó debug/tracing (event đi qua nhiều broker).
 - ✓ Quản lý event ordering, idempotency phức tạp.
 - **Ví dụ:** Stock trading, fraud detection, IoT telemetry.
-

4. CQRS (Command Query Responsibility Segregation)

- **Khái niệm:** Tách biệt phần xử lý ghi (commands) và đọc (queries). Thường kết hợp với event sourcing.
 - **Ưu điểm:**
 - ✓ Tối ưu hiệu năng khi read/write có nhu cầu khác nhau.
 - ✓ Cải thiện khả năng mở rộng (read scale lớn).
 - ✓ Dễ dàng tạo view riêng cho báo cáo/analytics.
 - **Thách thức:**
 - ✓ Đồng bộ dữ liệu giữa read/write model khó.
 - ✓ Tăng độ phức tạp thiết kế.
 - **Ví dụ:** E-commerce order tracking, mạng xã hội (feed).
-

5. Event Sourcing

- **Khái niệm:** Thay vì lưu trạng thái cuối cùng, hệ thống lưu toàn bộ chuỗi sự kiện. Trạng thái được “replay” từ events.
 - **Ưu điểm:**
 - ✓ Audit trail đầy đủ, dễ rollback.
 - ✓ Hỗ trợ undo, versioning.
 - ✓ Tích hợp tốt với CQRS.
 - **Thách thức:**
 - ✓ Storage lớn, dễ phình dữ liệu.
 - ✓ Phải có cơ chế snapshot để tối ưu.
 - **Ví dụ:** Ngân hàng (giao dịch), hồ sơ y tế.
-

6. Hexagonal Architecture (Ports & Adapters)

- **Khái niệm:** Ứng dụng chia thành *core domain* và các *adapters*. Core giao tiếp ra ngoài qua *ports* (giao diện trừu tượng). Adapters triển khai ports này (ví dụ: adapter DB, adapter UI).
 - **Ưu điểm:**
 - ✓ Domain độc lập với hạ tầng.
 - ✓ Test dễ (mock adapters).
 - ✓ Thay đổi công nghệ (DB, UI) mà không ảnh hưởng core.
 - **Thách thức:**
 - ✓ Overhead với hệ thống nhỏ.
 - ✓ Cần thiết kế rõ ràng từ đầu.
 - **Ví dụ:** Payment gateway, ERP core.
-

7. Clean / Onion / Domain-Driven Design (DDD) Architecture

- **Khái niệm:** Phân chia thành nhiều lớp (domain, application, infrastructure). Domain model nằm ở trung tâm, không phụ thuộc bên ngoài.
 - **Ưu điểm:**
 - ✓ Rất phù hợp với domain phức tạp.
 - ✓ Giảm coupling, tăng maintainability.
 - ✓ Ngôn ngữ chung (Ubiquitous Language) giữa dev và business.
 - **Thách thức:**
 - ✓ Đòi hỏi kỷ luật, dễ bị “over-engineering”.
 - ✓ Mất thời gian ban đầu để mô hình hóa.
 - **Ví dụ:** Logistics, bảo hiểm, supply chain.
-

8. Modular Monolith

- **Khái niệm:** Monolith nhưng thiết kế module hóa, biên giới rõ ràng. Có thể tách dần thành microservices khi cần.
 - **Ưu điểm:**
 - ✓ Ít phức tạp hơn microservices.
 - ✓ Module hóa giúp dễ bảo trì.
 - ✓ Bước trung gian tốt trước microservices.
 - **Thách thức:**
 - ✓ Nếu coupling cao, sẽ thành monolith “xấu”.
 - ✓ Không tận dụng được scale độc lập.
 - **Ví dụ:** Startup SaaS (billing, reporting, auth).
-

9. Reactive / Streaming Architecture

- **Khái niệm:** Dựa trên reactive programming và stream processing. Hệ thống phản ứng khi dữ liệu mới đến.
- **Ưu điểm:**

- ✓ Xử lý real-time (IoT, analytics, big data).
 - ✓ Hỗ trợ backpressure, khả năng chịu tải cao.
 - **Thách thức:**
 - ✓ Thiết kế phức tạp.
 - ✓ Cần hạ tầng stream (Kafka, Pulsar, Flink).
 - **Ví dụ:** Clickstream analysis, video analytics, fraud detection.
-

10. Edge / Fog Computing

- **Khái niệm:** Đưa tính toán & lưu trữ đến gần nguồn dữ liệu (thiết bị IoT, edge node), thay vì chỉ cloud trung tâm.
 - **Ưu điểm:**
 - ✓ Độ trễ thấp hơn, phù hợp real-time.
 - ✓ Giảm băng thông gửi lên cloud.
 - ✓ Tăng tính sẵn sàng khi kết nối cloud kém.
 - **Thách thức:**
 - ✓ Quản lý phân tán, đồng bộ dữ liệu khó.
 - ✓ Hạn chế tài nguyên edge devices.
 - **Ví dụ:** Smart city, cảm biến công nghiệp, xe tự hành.
-

Như vậy, các kiến trúc hiện đại tập trung vào: **phân tán – linh hoạt – real-time – tích hợp cloud/IoT**, đáp ứng nhu cầu phần mềm ngày nay khác biệt so với kiến trúc truyền thống.